Da Teoria ao Código: Implementando *K-Armed Bandits* com Padrões de Projeto.

Autor: Marco Aurélio Guimarães Falcão – Engenheiro de Sistemas

Data: 10/08/2026

1. Introdução

Este artigo trata da implementação em Python de algoritmos de solução para um problema clássico de Aprendizado de Reforço, o problema dos K-bandidos descrito no Capítulo 2 do Livro de [Sutton & Barto] [1]. Esse problema introduz os conceitos mais básicos de Aprendizado de Reforço, como por exemplo: Ambiente, Agente, Ações, Políticas, e Recompensas. O livro [1] também apresenta as principais estratégias de solução do problema dos K-bandidos: Método de Estimativa do Valor da Ação, Valor Inicial Otimista, Limite Superior de Confiança (UCB – *Upper Confidence Bound*), e Método dos Gradientes.

O Capítulo 1 do Livro [Lattimore & Szepesvári] aborda o conceito de Arrependimento como uma métrica de medida de desempenho do aprendizado do agente bastante utilizada na comparação do uso de um mesmo algoritmo em diferentes ambientes. A seção 36.1 do mesmo livro aborda a estratégia de solução por *Thompson-Sampling* (TS), uma alternativa de melhor desempenho mas com maior custo computacional.

Para implementação das Estratégias de solução anteriormente citadas foram utilizados os padrões de projeto ***Strategy*** e ***Template Method***. O primeiro possibilita o fácil reuso dessas soluções em diversos casos de teste, enquanto o segundo permite abstração e utilização de uma interface padronizada, otimizando a implementação da hierarquia de classes de solução.

Foram utilizados alguns exemplos do Capítulo 2 do Livro de [Sutton & Barto] [1] como casos de teste para validação da implementação em Python nos quais foram obtidos resultados semelhantes para ambientes estacionários. Além disso, foram explorados casos de teste para ambientes não estacionários como no exercício 2.11 desse livro.

2. Apresentação Geral de RL (*Reinforcement Learning*) e Definição do Problema.

A Figura 1 apresenta o Esquema Geral do Aprendizado por Reforço, ilustrando a interação cíclica entre um Agente e um Ambiente. O Agente executa ações que impactam o Ambiente; este, por sua vez, reage fornecendo observações — que podem indicar seu estado interno de forma direta ou indireta — e recompensas, que servem como métricas de sucesso ou fracasso para as ações tomadas. O Agente processa esse *feedback* ao longo

do tempo, buscando estimar o valor esperado (ou valor médio) da recompensa para cada ação. Com base nessas estimativas e no estado atual do Ambiente, o Agente determina a próxima ação a ser tomada, seguindo uma estratégia ou política definida.

O Arrependimento é a perda acumulada de recompensa sofrida pelo agente ao longo do tempo, decorrente da escolha de ações subótimas em vez da ação de maior valor esperado (ação ótima).

De forma intuitiva, o Aprendizado por Reforço pode ser comparado ao processo de educar um filhote de cachorro ao chegar em um novo lar. Inicialmente, movido pela exploração, o animal realiza diversas ações aleatórias: rói o pé da cadeira, destrói meias ou faz suas necessidades em locais inadequados. O aprendizado ocorre através da intervenção do tutor, que atua como o mediador das recompensas. A cada comportamento desejado, o filhote recebe um reforço positivo (como um biscoito); para cada ação inconveniente, recebe um reforço negativo (como uma chamada de atenção). Com o tempo e a repetição, o pequeno animal integra esse histórico de *feedback* e aprende a selecionar as ações que maximizam suas recompensas, compreendendo as 'regras' do ambiente da casa.

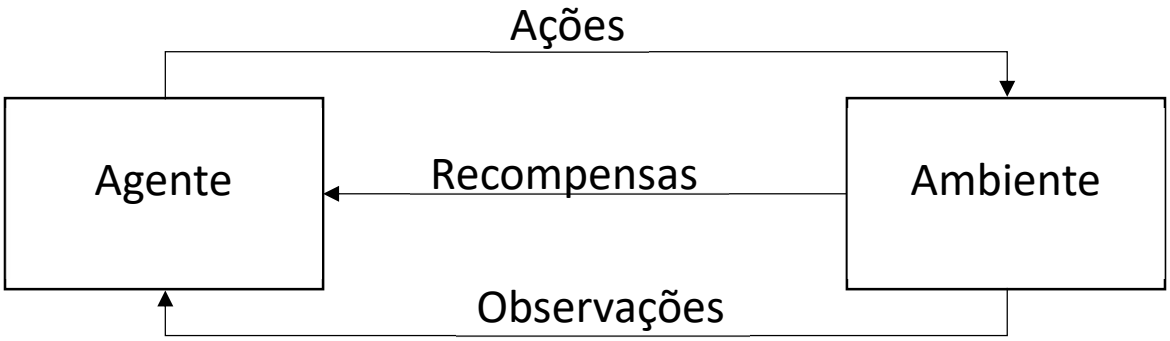


Figura 1 – Esquema Geral do Aprendizado por Reforço

A estratégia de solução do problema de Aprendizado Por Reforço aborda “Como” escolher a melhor ação e é conhecida como Política. Uma solução ***Greedy* (Gulosa)** procura sempre escolher a ação com maior recompensa média. Essa técnica é chamada de **explotação**. Para evitar que o Agente fique 'preso' em uma escolha apenas razoável, é fundamental permitir a **exploração**: a prática de experimentar, ocasionalmente, outras ações com médias inferiores ou desconhecidas. Um exemplo cotidiano desse dilema é o hábito de frequentar sempre o mesmo restaurante por já conhecer seu custo-benefício (explotação) versus a decisão de, ocasionalmente, visitar um local novo para descobrir sabores superiores (exploração).

O Arrependimento é a métrica ideal para avaliar algoritmos em simulação porque captura o custo da exploração de forma mais precisa que a recompensa bruta. Porém, seu cálculo é inviável no mundo real, no qual não conhecemos a recompensa da ação ótima. Como na vida, o arrependimento é aquele sentimento de experimentar um restaurante novo e perceber que deveria ter ido no de sempre.

Para a compreensão inicial do problema, recorre-se à etimologia de *'One-Armed Bandit'* [3] (o 'Bandido de um Braço Só'). O termo deriva da junção de *one-armed* (referindo-se à alavanca única que opera as máquinas caça-níqueis clássicas) e *bandit* (um ladrão que atua em áreas sem lei ou alguém que engana e lesa outros). Essa metáfora reflete o fato de que tais máquinas são projetadas para 'roubar' o capital dos jogadores que, estatisticamente, perdem para a banca ao longo do tempo.

O problema dos **K-Bandidos** é uma extensão matemática do dilema do *'One-Armed Bandit'* para um cenário com K máquinas caça-níqueis. Nesse modelo, um operador (ou **Agente**) busca identificar e selecionar a alavanca da máquina que proporciona o maior retorno médio de recompensas ao longo do tempo, conforme ilustrado na Figura 2. O desafio central reside no fato de que as distribuições de probabilidade de recompensas de cada máquina são inicialmente desconhecidas, exigindo que o Agente equilibre a exploração de novas máquinas com a explotação daquelas que já demonstraram bons resultados. O problema dos K bandidos também é conhecido na literatura como MAB (*Multi-Armed Bandits Problem*).

Ao descrever o problema de Aprendizado por Reforço, é fundamental distinguir os dois tipos de ambientes considerados: o **Estacionário** e o **Não Estacionário**. No ambiente estacionário, as propriedades estatísticas das recompensas (como a média e o desvio padrão de das recompensas de cada braço) permanecem constantes ao longo do tempo. Já no ambiente não estacionário, essas estatísticas mudam com o tempo, exigindo que o agente seja capaz de adaptar suas estimativas continuamente, alterando e atualizando a Política de forma dinâmica. Esta distinção é crucial, pois dita a escolha da estratégia de atualização e o peso dado às recompensas mais recentes.

Uma Política π define como uma ação $a \in A$ deve ser escolhida em função das observações obtidas que definem o estado s do Ambiente em questão, como mostrado na Equação (1). Ou seja, a Política é traduzida pelo conjunto de probabilidades da escolha da ação a pertencente ao conjunto de ações possíveis *A* dado que o ambiente se encontra no estado s.

$$\pi = \text{Prob}(a \in A/s) \quad (1)$$



Figura 2 – O Problema dos K Bandidos – Gerada pelo Gemini 3 Flash Image

O problema dos K-Bandidos é considerado a porta de entrada mais acessível para o Aprendizado por Reforço por ser um modelo de estado único. Diferente de problemas mais complexos, aqui o ambiente não muda de configuração (não muda de estado) após uma ação; o foco reside na estimativa das recompensas e na otimização da política de escolha. Ambientes mais sofisticados exigem o tratamento de mudanças de estado probabilísticas como os Processos de Decisão de Markov (MDP) ou consideração de restrições de custo ou esforço para realizar a ação, que fogem ao escopo deste texto.

Suponha que a máquina 3 das 10 máquinas da Figura 2 possua a melhor média recompensas μ^* em relação a todas as máquinas. A política ótima π^* seria definida pelo conjunto de probabilidades mostrado em (2). Ou seja, ao jogar sempre na máquina 3 obtêm-se a melhor recompensa média. O agente desconhece qual a melhor máquina então ele procura aprender a partir das recompensas qual a melhor estratégia para escolha da ação, nesse caso, a máquina a escolher.

$$\pi * = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0] \quad (2)$$

O problema dos K-bandidos possui várias aplicações práticas a destacar, entre elas [2][4]:

- Recomendação de Conteúdo – apresentação de artes personalizadas dos filmes para cada usuário pelo Netflix. As melhores artes vão captar a atenção dos clientes para assistir filmes ou séries [5];
- Testes A/B adaptativos – escolha de melhor arranjo de componentes gráficos (*Widgets*) em uma página Web que resulte em melhores resultados, i.e., mais compras realizadas pelo cliente;
- Ensaio Clínicos - dosagem de medicamentos com menor custo ético. Permite aos praticantes da Medicina experimentar tratamentos dos pacientes com medicamentos observando seus resultados;
- Redes de Drones – otimização de trajetórias e alocação de recursos (potência e bateria) em missões de resgate. Em uma área de desastre a infraestrutura da rede celular é destruída e um drone pode ser utilizado como ERB (Estação Rádio-Base) para estabelecer comunicação com as vítimas. Nesse caso os bandidos são um número discretos de direções possíveis do drone assumir, para cada direção são medidas Recompensas em termos de melhor comunicação com as vítimas sujeitas a restrição de custo de gasto de bateria e potência de transmissão. Em função dessas restrições o número de direções possíveis é alterado de forma dinâmica [6]; e
- Finanças – Seleção dinâmica de portfólios e estratégias de *trading* de ações que se adaptam rapidamente a mudanças de regime no mercado. Numa estratégia de investimento mensal de novos aportes ou dividendos escolher em que ativo da carteira investir.

Este estudo foi desenvolvido por meio de um processo colaborativo com diversas Inteligências Artificiais (IAs) usando recursos livres, abrangendo desde o brainstorming conceitual e a estruturação do artigo até a elaboração, revisão e correção de código, definição da arquitetura de classes, geração de imagens e indicação de referências. Optou-se deliberadamente por não utilizar automações de agentes para geração integral e automática do texto. Em vez disso, o autor orquestrou a construção do conteúdo através de diálogos iterativos com diferentes modelos, priorizando dois objetivos fundamentais: o aprofundamento técnico no tema de Aprendizado por Reforço e a exploração das capacidades e nuances das próprias IAs utilizadas. Para este estudo foram usadas duas abordagens: construção incremental (*Build-a-little Test-a-Little*) e *Human-in-the-Loop* para elaboração de perguntas e testes/validação das respostas das IAs. As ferramentas específicas de IA empregadas estão devidamente citadas na Conclusão.

A linguagem Python é utilizada para a implementação das soluções. Por ser a linguagem predominante na área de IA, ela dispõe de bibliotecas que facilitam a codificação, como o pacote ***Gymnasium*** (ou Gym) [7], que padroniza a interface entre o agente e o ambiente.

Complementarmente, o uso do **Google Colab** possibilita o desenvolvimento do código de forma progressiva e colaborativa, oferecendo suporte de GPU e integração com a IA Gemini, o que garante maior agilidade na execução dos testes e experimentos realizados.

3. **Estratégias de Solução:**

As estratégias de solução baseiam-se na estimativa do valor esperado das recompensas para cada ação (alavanca), conforme definido pela Média Amostral (3).

Seja R_i a recompensa obtida após o i-ésimo acionamento de uma determinada alavanca: O valor estimado da ação após ela ter sido selecionada n-1 vezes, denotado por Q_n, é calculado pela média aritmética das recompensas recebidas, com i variando de 1 até n-1

$$Q_n = \frac{R_1 + R_2 + \ldots + R_{n-1}}{n-1} \quad (3)$$

Conforme a Lei dos Grandes Números, à medida que n aumenta, a estimativa Qn converge para o valor real da recompensa média da alavanca q*.

A representação recursiva apresentada na Equação (4) é matematicamente equivalente à média aritmética, porém mais eficiente para implementação computacional. Ela isola o **erro de estimativa** — a diferença entre a recompensa atual e a média anterior — e o escala por um fator de passo (1/n), permitindo que a estimativa convirja de forma incremental para o valor real q*.

$$Q_{n+1} = Q_n + \frac{1}{n}[R_n - Q_n] \quad (4)$$

A Equação (4) pode ser interpretada como uma regra de atualização baseada no erro de estimativa, onde o novo valor é ajustado a partir da média anterior somada a um incremento ponderado. Esse incremento, representado pelo termo (R_n - Q_n), é tecnicamente denominado **erro de predição** ou **inovação**.
A estrutura geral dessa atualização é expressa na Equação (5), na qual o peso variável 1/n é substituído por um parâmetro de tamanho de passo (ou taxa de aprendizagem) constante α.

$$Q_{n+1} = Q_n + \alpha[R_n - Q_n] \quad (5)$$

Uma vez calculadas as estimativas por meio das Equações (4) ou (5), o agente deve selecionar a próxima ação a ser executada. O critério mais comum é o método ***Greedy* Puro (Guloso)**, no qual o agente seleciona a opção que apresenta a maior estimativa de Valor da Ação no momento atual. Formalmente, a ação selecionada no passo t, denotada por A_t, é definida como:

$$A_t = argmax_{(a \in A)}[Q_t(a)] \quad (6)$$

Entretanto, as estimativas de valor da ação são variáveis estatísticas que apresentam desvios em relação à média real q*. Pelo **Teorema do Limite Central**, o erro dessas estimativas tende a uma distribuição normal com média 0 e desvio padrão dado por $\frac{\sigma}{\sqrt{(n)}}$. Consequentemente, a escolha da ação com a melhor estimativa atual não garante o melhor desempenho a longo prazo, dado que o valor de $Q_t(a)$ pode estar momentaneamente distorcido por flutuações amostrais.

Para mitigar esse risco, utiliza-se a estratégia **ε-Greedy**. Neste método, o agente alterna entre:

- **Explotação (*Exploitation*):** Com probabilidade 1-ε, seleciona-se a ação de maior valor estimado.
- **Exploração (*Exploration*):** Com probabilidade ε, seleciona-se uma ação aleatória dentro do espaço $\mathcal{A}$ de ações, permitindo ao agente refinar suas estimativas sobre ações potencialmente subestimadas.

Nota-se que o Valor de Ação para cada bandido deve partir de um valor inicial no instante 0, Q_0. É comum inicializar Q_0 igual a 0 para todos os K bandidos. Entretanto, há uma solução que inicializa Q_0 com valores mais altos e iguais para todos os K bandidos. Tal solução é conhecida como **Valor Inicial Otimista** (*Optimistic Initial Value*) e utiliza a regra estabelecida pela Equação (6) para explorar todos os K bandidos logo no início do algoritmo.

Isso possibilita obter melhor desempenho através de uma exploração sistemática e completa de todas as ações, o que geralmente acelera a identificação da ação ótima em problemas **estacionários**, em relação ao método ***Greedy* Puro**. Contudo, como o método **ε-Greedy** já possui uma exploração implícita, **não é recomendado** usar Valor Inicial Otimista com ele, pois essa combinação pode saturar a exploração e até piorar o desempenho da solução.

Em ambientes não estacionários, essa técnica perde eficiência, uma vez que a exploração inicial se mostra insuficiente para acompanhar as mudanças do sistema ao longo do tempo. Nesses cenários, observa-se uma melhora ao adotar um tamanho de passo constante mais elevado, o que permite ao algoritmo priorizar as recompensas recentes em detrimento das estimativas históricas. No entanto, os experimentos realizados neste estudo revelam que esse ganho ocorre apenas nos passos iniciais, conforme detalhado na seção de resultados.

No presente artigo e no capítulo 2 do Livro Sutton & Barto [1] as recompensas do ambiente estacionário para cada Ação no instante n, Rn, são geradas a partir de distribuições normais com variância unitária, cujas médias são, por sua vez, sorteadas de uma distribuição normal padrão - $\mathcal{N}$ ($\mathcal{N}$(0,1), 1), ou seja, as médias de recompensa para cada bandido μ1, μ2,..., μk têm média zero e desvio padrão unitário, e as recompensa em si, tem média μi e desvio padrão unitário para cada bandido i $\in$ [1, K]. Os valores esperados (médias reais) de recompensa para cada braço, μ1, μ2,.., μk são amostrados inicialmente de uma distribuição normal padrão $\mathcal{N}$ (0, 1) e não mudam mais, de forma que o máximo de μi, μ* pela Equação (7), já indica o bandido que resulta em melhores recompensas médias, o que é útil para avaliar o percentual de escolhas ótimas do algoritmo.

$$\mu_* = max_{(a \in A)}[\mu_1, \mu_2, \ldots, \mu_k] \tag{7}$$

Para o ambiente Não Estacionário as médias das Normais utilizadas para sorteio e geração das recompensas são modificadas em cada passo pela soma de um desvio obtido de uma distribuição Normal *N*(0, 1) como no Equação (8).

$$[\mu_1, \mu_2, \ldots, \mu_k] += N(0,1) \tag{8}$$

Para o cálculo do percentual de escolha ótima, adota-se como referência a alavanca de maior média inicial, fundamentada na expectativa teórica do sistema. Visto que as perturbações adicionadas a cada passo são originárias de distribuições normais de média nula, a esperança matemática de longo prazo das recompensas permanece inalterada, conservando a identidade da alavanca ótima original. Dessa forma, a métrica mensura a robustez do algoritmo em convergir para a melhor escolha teórica mesmo sob o acúmulo de flutuações estocásticas ao longo do tempo.

Vimos como funciona o método **ε-Greedy**, que pode ser considerado análogo a uma **fila de prioridades**. Nela, a prioridade é baseada na estimativa de Valor da Ação (Q) para cada alavanca, recalculada a cada passo do algoritmo para o bandido selecionado. Eventualmente, um "juiz" realiza um sorteio entre todos os braços para decidir quem deve passar à frente da fila, baseando-se em uma probabilidade fixa ε.

Dessa forma, a regra de exploração é fundamentada no par **(Q, ε)** , sendo Q a Estimativa do Valor da Ação para cada braço e ε um valor fixo geralmente pequeno. A grande limitação dessa abordagem é que uma alavanca promissora pode, por simples falta de sorte, demorar muito a ser selecionada novamente, tornando a exploração pouco efetiva e sujeita ao esquecimento de ações potencialmente ótimas.

Continuando com a analogia, o método **UCB (*Upper Confidence Bound*)** procura usar informações adicionais para definir a prioridade da fila, como o tempo decorrido t – equivalente ao número de passos do algoritmo, e o número de vezes que cada alavanca é acionada N. Dessa forma a regra de prioridade é estabelecida com base na tupla **(Q, t, N).**

O Teorema do Limite Central estabelece que o desvio padrão em relação a estimativa da média é igual a $\frac{\sigma}{\sqrt{N}}$ e o intervalo de confiança em torno da estimativa da média corresponde a $\pm c.\frac{\sigma}{\sqrt{N}}$ em que c é uma constante. O limite superior do intervalo de confiança é igual a $Q + c.\ \frac{\sigma}{\sqrt{N}}$. Partindo dos Fundamentos em Limites de Probabilidades - Desigualdade de Hoeffding, o método UCB estabelece através da equação (9) um limite superior de intervalo de confiança para as estimativas das médias de valor das ações para cada alavanca de forma a aumentar as chances de ela ser selecionada quanto maior for o tempo decorrido t e quanto menor for o número de vezes dela ter sido selecionada no passado Nt(a). Assim, consegue uma melhor exploração das alavancas contornando o problema do possível esquecimento de uma alavanca no método **ε-Greedy.**

$$A_t = argmax_{(a \in A)}[Q_t(a) + c.\sqrt{\frac{log(t)}{N_t(a)}}] \quad (9)$$

Obs: c = 1 ou c = 2 são valores tipicamente usados com a equação (9).

A Figura 3 apresenta o Diagrama de Blocos do Método do Gradiente. Esse método baseia-se na busca por uma política ótima π, que se inicia como uma distribuição uniforme (onde todos os K bandidos são equiprováveis), sendo atualizada a cada ciclo com base em um índice conhecido como H (Preferência da Ação). Essa atualização das preferências pode ser orientada pelo desvio da Recompensa Atual em relação à média das recompensas passadas (com Baseline) ou basear-se puramente no valor bruto da Recompensa Atual

(sem Baseline). Por fim, o vetor H é mapeado através da função Softmax (H) para gerar a nova distribuição de probabilidades da política π para o ciclo seguinte.

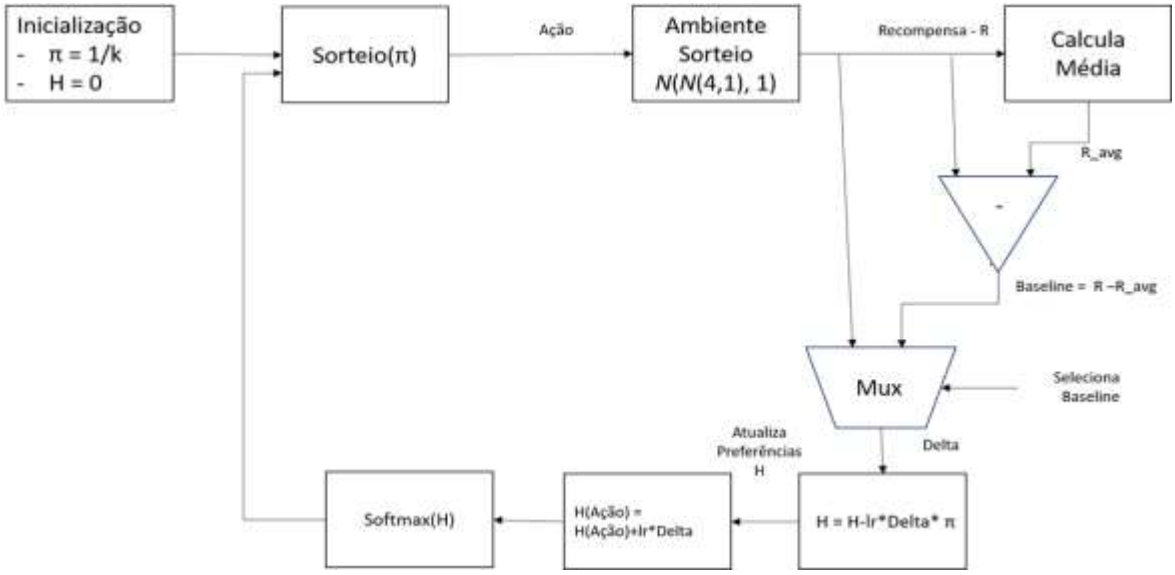


Figura 3-Diagrama de Blocos do Método do Gradiente

Obs: O hiperparâmetro do Método do Gradiente é a taxa de aprendizado lr (*Learning Rate*).

A seção 36.1 da página 461 da referência [2] apresenta a estratégia de solução do problema dos K bandidos armados por *Thompson-Sampling* (TS). O TS é uma estratégia probabilística para o problema dos K bandidos que estima, para cada ação, os parâmetros de média e variância (ou desvio-padrão) da distribuição de recompensas do ambiente. Inicialmente, assume-se um conhecimento a priori, atribuindo os mesmos valores iniciais de média e desvio-padrão a todas as ações. A cada passo, amostra-se valores da distribuição a priori para cada ação e seleciona-se aquela que maximiza o valor amostrado. Ao acionar a alavanca escolhida, observa-se uma recompensa, que é então utilizada para atualizar os parâmetros da distribuição a posteriori por meio do Teorema de Bayes. Essa distribuição atualizada torna-se o novo prior para o passo seguinte, e o processo é repetido ao longo do horizonte temporal. Com isso, o TS converge progressivamente para uma boa estimativa das distribuições reais de recompensa do ambiente.

O agente *Thompson Sampling* (TS) empregado neste artigo baseia-se no uso de distribuições normais para modelar a crença sobre a recompensa média de cada ação. Esta abordagem mostrou-se adequada para os cenários investigados, nos quais o

ambiente também gera recompensas a partir de distribuições normais. Embora o TS com prior normal funcione satisfatoriamente em uma ampla gama de problemas — graças à robustez da atualização Bayesiana e ao Teorema Central do Limite — há situações em que outras distribuições são mais apropriadas. Exemplos incluem o modelo Beta-Bernoulli para recompensas binárias, a distribuição Gamma para recompensas positivas assimétricas, a t-*Student* para caudas pesadas, a Poisson para dados de contagem, e abordagens empíricas ou baseadas em *bootstrap* para distribuições desconhecidas ou multimodais. A escolha da distribuição adequada deve considerar a natureza das recompensas observadas e o desempenho desejado em cada aplicação.

As complexidades computacionais estimadas para cada algoritmo (Tabela 1) foram consolidadas a partir de uma análise de algoritmos, baseada na estrutura de seus loops de decisão e atualização (por exemplo, ε-*greedy* e UCB possuem operações O(1) por passo, enquanto os algoritmos Gradiente e *Thompson Sampling* envolvem varreduras O(K)). Essa análise foi refinada durante uma discussão aprofundada com um assistente de IA (DeepSeek-V4-Flash, 2026) para fins de clareza e comparação no contexto deste trabalho.

Agentes	Custo por Passo	1000 steps x 2000 runs
ε-Greedy	O(1) – atualização simples	1 x (baseline)
UCB	O(1) – cálculo simples	~1,2x
Gradiente	O(k) – atualiza todas as ações	~2-3x(depende do k)
TS(original)	O(k+n) – com np.var() no histórico para um número de acionamentos n da alavanca	~5-10x
TS (otimizado)	O(k) - Wellford	~2-3x

Tabela 1 - Comparação de Custos da Estratégias de Solução

Obs: A Solução de Wellford para *Thompson Sampling* ao invés de usar todas as amostras para cálculo da variância a posteriori processa cada amostra de forma incremental.

Como veremos na seção de Resultados, o *Thompson Sampling* (TS) apresenta um desempenho superior a maioria das demais estratégias de solução, tanto em recompensa acumulada quanto em taxa de acerto da ação ótima. Este ganho é atribuído à sua capacidade de equilibrar exploração e explotação de forma inerentemente adaptativa. Entretanto, esse benefício vem acompanhado de um maior custo computacional, especialmente quando implementado com distribuições mais complexas (como Gamma, t-Student ou *bootstrap* (sorteio com reposição de um conjunto de amostras)), que exigem operações adicionais por passo. Apesar disso, em cenários com horizonte temporal

suficientemente longo ou com recursos computacionais adequados, o TS se mostra uma escolha atraente e robusta.

Embora o *Thompson Sampling* tenha apresentado o melhor desempenho em termos de recompensa acumulada na maioria dos casos, o UCB (*Upper Confidence Bound*) permanece como uma alternativa amplamente utilizada em aplicações práticas devido à sua melhor relação custo-benefício. O UCB combina um desempenho próximo ao ótimo com um custo computacional significativamente menor, além de uma implementação mais simples e ajuste de hiperparâmetros menos sensível. Além disso, o UCB conseguiu superar o TS com a escolha de um hiperparâmetro c apropriado.

4. **Métricas de Avaliação das Soluções:**

A Tabela 2 apresenta as métricas utilizadas na avaliação dos casos de teste. Essa avaliação é feita pela interpretação dos gráficos gerados em cada execução.

Para avaliar o desempenho dos algoritmos sob uma perspectiva multifacetada, os experimentos foram analisados a partir de três métricas complementares, cujas características, requisitos e aplicabilidades estão sintetizados na **Tabela 2**. Enquanto a recompensa média por passo fornece um índice de ganho relativo entre as abordagens, o percentual de escolha ótima avalia a precisão estrita do aprendizado. Complementarmente, o *Regret* acumulado quantifica o custo decorrente das decisões subótimas tomadas ao longo do horizonte temporal, permitindo avaliar as perdas de cada política de exploração.

Tipo de métrica	**Recompensa Média por passo em um número de corridas**	**% Escolha Ótima**	***Regret* Acumulado**
Melhor adequação	Análise comparativa Entre métodos em um mesmo ambiente	Resultado absoluto/ Praticidade imediata	Pesquisa/Ajuste Fino de algoritmos/ Otimização
Conhecer μ_* (requer modelagem/identificação a priori do ambiente)	Não necessário	Sim	Sim
Comparação entre estratégias de solução para um mesmo ambiente	Sim	Sim	Sim

Comparação da mesma estratégia de solução em ambientes diferentes	Não	Sim	Sim
Mostra o custo do erro	Sim, inversamente	Não	Sim
Uso	O método A resulta em maior recompensa média por passo que o método B	90% das vezes acertou a escolha ótima	Perdeu 500 unidades

Tabela 2 – Métricas de Avaliação das Soluções

Obs: μ*- maior média estatística das recompensas entre as k alavancas, correspondente a escolha ótima.

Seja R_t a recompensa obtida pelo agente no passo t (para $t = 1,2, \dots, n$), onde n é o número total de passos (horizonte temporal). A **Recompensa Média Acumulada** $\bar{R}_T$ é definida como a média aritmética de todas as recompensas obtidas até o passo T:

$$\bar{R}_T = \frac{1}{n}\sum_{t=1}^{T} R_t \tag{10}$$

A Porcentagem (%) de Escolha Ótima pode ser calculada pela razão entre o número de vezes que o bandido ótimo foi escolhido N(Q*) pelo número de passos de execução do algoritmo (num_steps).

$$\%_\text{Escolha_ó}tima = \frac{N(Q*)}{num_steps} \tag{11}$$

O **Arrependimento (regret) Acumulado** após n passos é definido como a diferença entre a recompensa total que seria obtida ao selecionar sempre a ação ótima (com média μ*) e a recompensa total efetivamente obtida pelo agente:

$$\boldsymbol{Regret(n) = n \cdot \mu_* - \sum_{t=1}^{n} R_t} \tag{12}$$

onde:

- μ* é a recompensa média da melhor ação (desconhecida pelo agente),
- R_t é a recompensa obtida no passo t.

Essa métrica é amplamente utilizada na literatura [Lattimore & Szepesvári, 2020] por capturar o custo da exploração de forma mais precisa que a recompensa bruta, embora seu cálculo seja inviável no mundo real, em que não conhecemos μ*.

5. **Arquitetura:**

A Figura 4 apresenta o Diagrama de Classes utilizado para execução dos casos de teste deste artigo através da implementação dos ambientes dos K-Bandidos (Estacionário e Não Estacionário) e dos Agentes para diferentes Estratégias de solução (*Alpha – Greedy* e *ε-Greedy*, UCB, *Gradient* e *Thompson Sampling* – TS - otimizado).

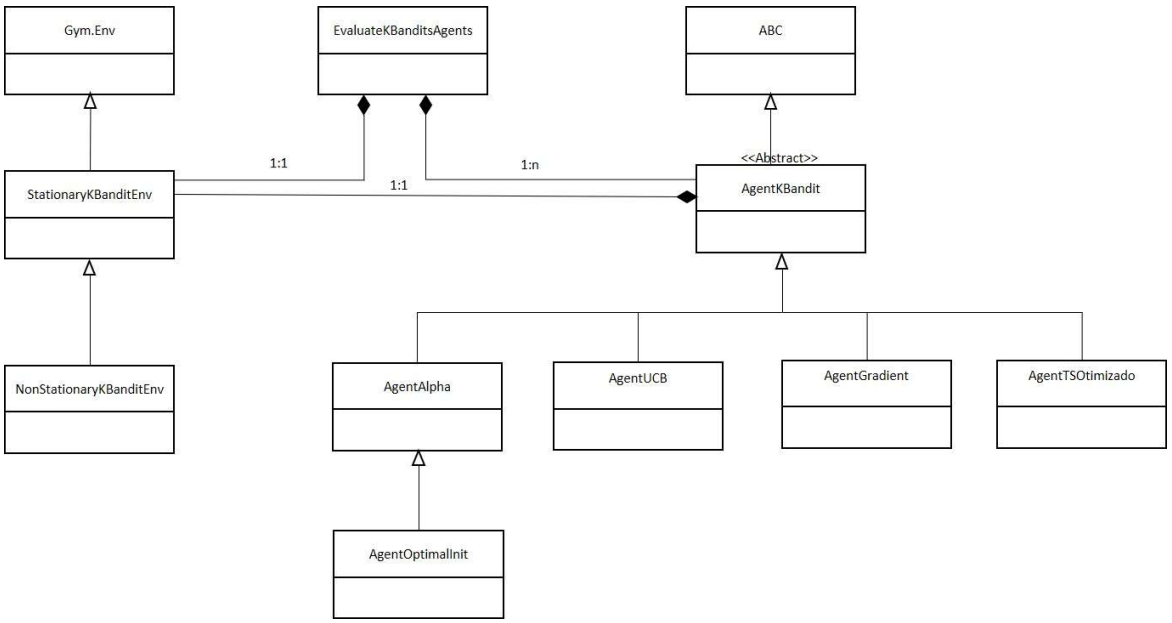


Figura 4-Diagrama de Classes para Implementação dos K Bandidos

O padrão de projeto *Strategy* [8] confere flexibilidade à arquitetura ao desacoplar o agente de controle e o ambiente. Essa abordagem facilita a execução de múltiplos cenários de teste e elimina a necessidade de estruturas condicionais complexas no código.

A aplicação do padrão Template Method [8] permite definir uma classe abstrata base para os agentes k -Bandidos, delegando o comportamento específico às subclasses

especializadas de cada estratégia. Essa estrutura preserva uma interface de chamada unificada, simplificando a alternância entre diferentes agentes de controle.

6. Resultados:

A Figura 5 ilustra a taxa de recompensa média e o percentual de escolha da ação ótima para os agentes ϵ-greedy ($\epsilon \in \{0; 0{,}01; 0{,}1\}$ com tamanho de passo $\alpha = 1/n$) e para o agente *Thompson Sampling* (TS) otimizado por meio do método de Welford. A simulação foi conduzida em um ambiente estacionário ao longo de 1.000 passos e 2.000 repetições (*runs*). Observa-se que a dinâmica dos agentes ϵ-greedy corrobora os resultados de referência [1, Fig. 2.2], ao passo que o agente TS otimizado superou o desempenho de todos os demais métodos avaliados.

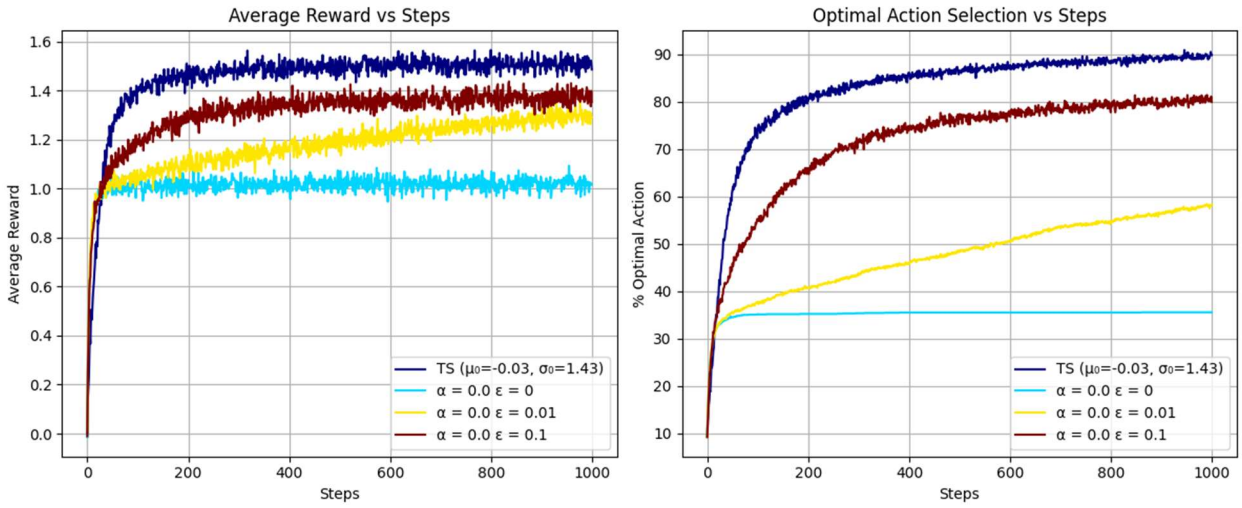


Figura 5- Resultados ε-greedy e TS - Ambiente Estacionário

A Figura 6 ilustra a taxa de recompensa média e o percentual de escolha da ação ótima para os agentes ϵ-greedy submetidos a diferentes inicializações — com valores iniciais otimistas ($Q_1 = +5, \epsilon = 0$) e realistas ($Q_1 = 0, \epsilon = 0{,}1$), ambos com tamanho de passo constante $\alpha = 0{,}1$ — em comparação ao agente Thompson Sampling (TS) otimizado pelo método de Welford. A simulação foi conduzida em um ambiente estacionário ao longo de 1.000 passos e 2.000 repetições (*runs*). Observa-se que a dinâmica dos agentes ϵ-greedy corrobora os resultados descritos em [1, Fig. 2.3], ao passo que o agente TS otimizado superou o desempenho de todos os demais métodos avaliados.

.

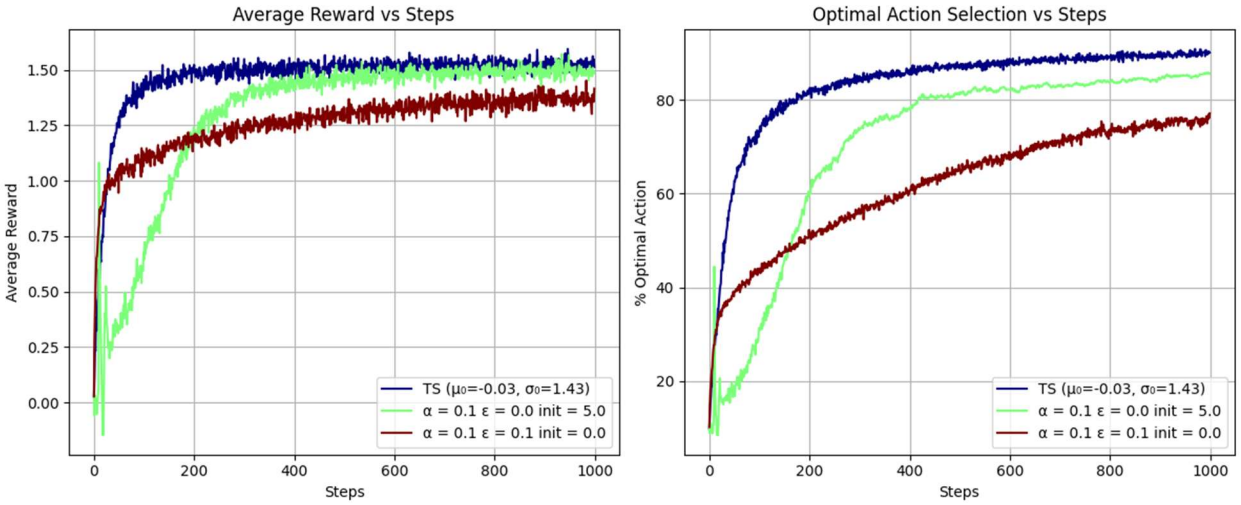


Figura 7 - Resultados ε-greedy com diferentes valores iniciais e TS - Ambiente Estacionário

A Figura 7 ilustra a taxa de recompensa média e o percentual de escolha da ação ótima para o agente ϵ-greedy ($\epsilon = 0{,}1$ com tamanho de passo $\alpha = 1/n$), para o agente UCB com constante de exploração $c = 2$ e para o agente Thompson Sampling (TS) otimizado pelo método de Welford. A simulação foi conduzida em um ambiente estacionário ao longo de 1.000 passos e 2.000 repetições (*runs*). Observa-se que o comportamento comparativo entre os agentes ϵ-greedy e UCB corrobora os resultados de referência [1, Fig. 2.4], ao passo que o agente TS otimizado superou o desempenho de todos os demais métodos avaliados.

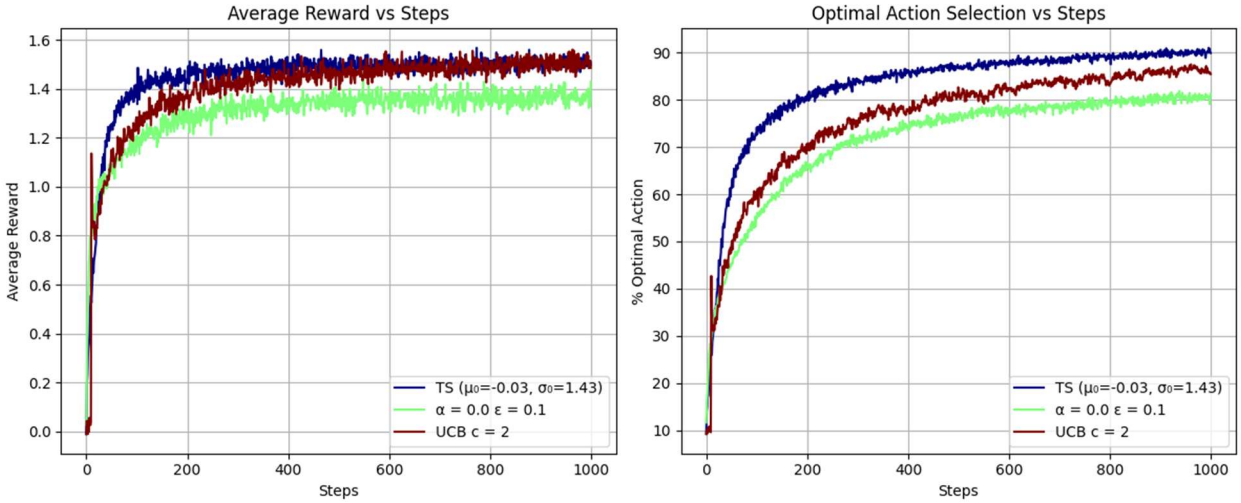


Figura 6- Resultados ε-greedy, UCB e TS - Ambiente Estacionário

A Figura 8 ilustra a taxa de recompensa média e o percentual de escolha da ação ótima para os agentes de Gradiente com e sem baseline (variando a Learning Rate $lr \in \{0{,}1; 0{,}4\}$, em comparação ao agente Thompson Sampling (TS) otimizado pelo método de Welford. A simulação foi conduzida em um ambiente estacionário ao longo de 1.000 passos e 2.000 repetições (*runs*). Observa-se que o comportamento comparativo entre os agentes de Gradiente corrobora os resultados de referência [1, Fig. 2.5], evidenciando o papel crucial da baseline no desempenho, ao passo que o agente TS otimizado superou o desempenho de todos os demais métodos avaliados.

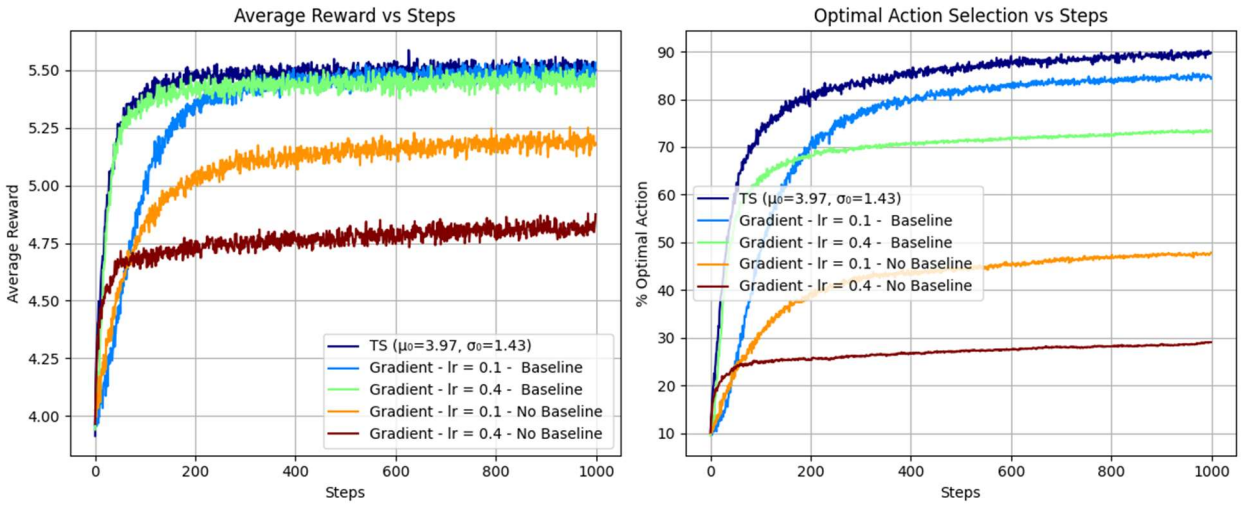


Figura 8 - Resultados Gradiente e TS - Ambiente Estacionário

A Figura 9 apresenta os mesmos resultados obtidos por Sutton & Barto [1, Fig. 2.6] para 1.000 passos e 2.000 repetições (*runs*), variando os hiperparâmetros em uma faixa de valores para os diversos agentes (com exceção do *Thompson Sampling*), o que valida a aplicação dessas estratégias em um ambiente estacionário. O algoritmo UCB obteve o melhor desempenho, alcançando mais de 80% de escolhas ótimas para $c = 1$.

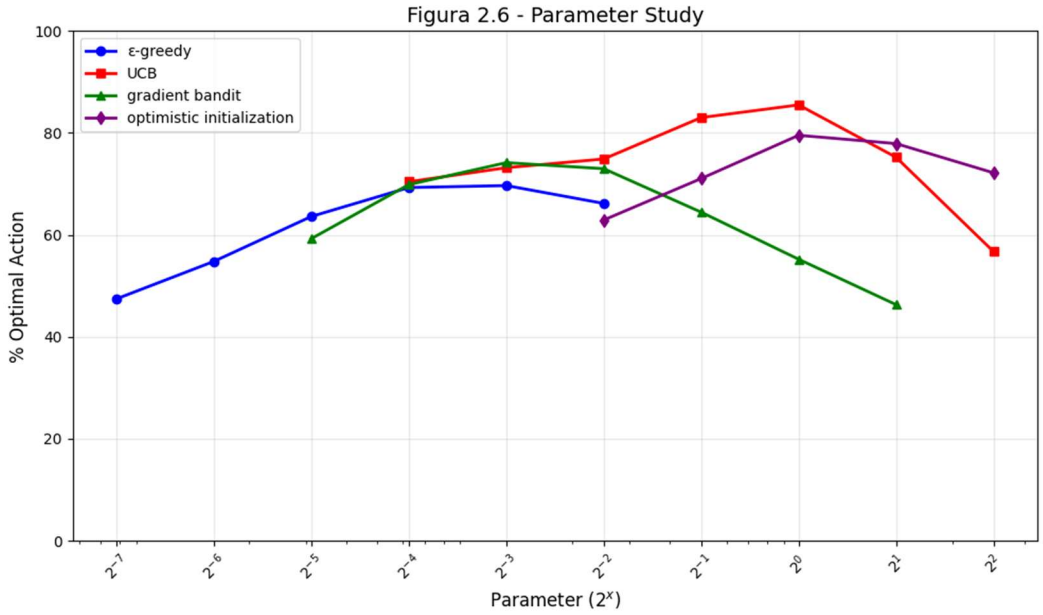


Figura 9- Variação de HiperParâmetros - Ambiente Estacionário

A Figura 10 apresenta a comparação entre o agente UCB ($c = 1$), o agente ε-greedy ($\varepsilon = 0{,}1$) e o agente *Thompson Sampling* para 1.000 passos e 2.000 repetições (*runs*) em um ambiente estacionário. Nesse cenário, o agente UCB superou o *Thompson Sampling* em desempenho.

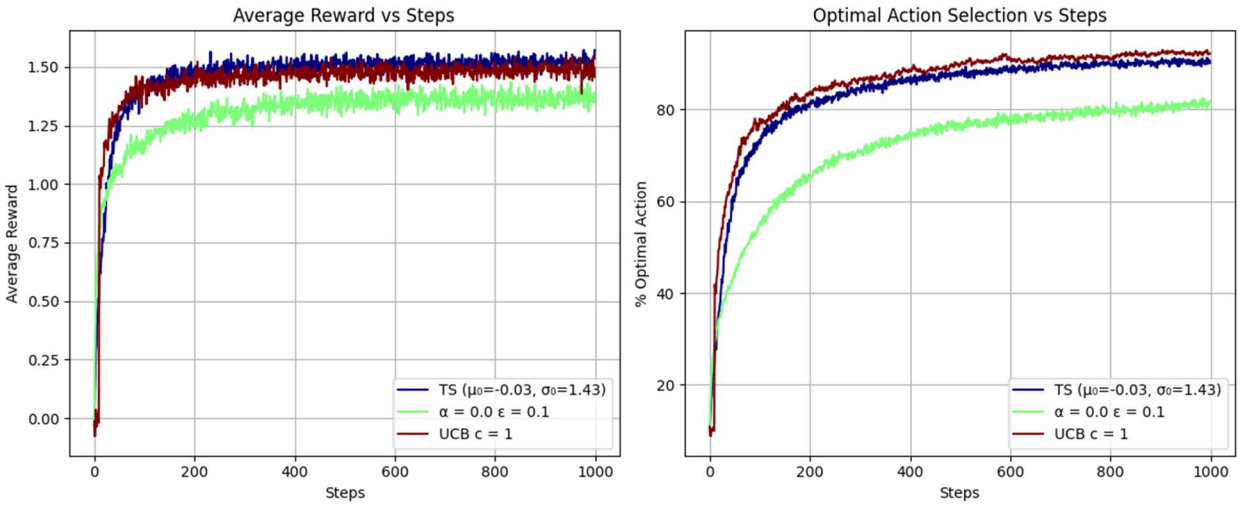


Figura 10- UCB com c = 1 e TS - Ambiente Estacionário

Conforme o enunciado do livro [1, Exercício 2.11], foram obtidos os resultados para todos os agentes — com exceção do *Thompson Sampling* — variando os hiperparâmetros para o cenário de ambiente não estacionário (Figura 11). O desempenho deteriorou-se em todos os agentes testados. O algoritmo UCB alcançou em torno de 51% de escolhas ótimas para $c = 1/8$.

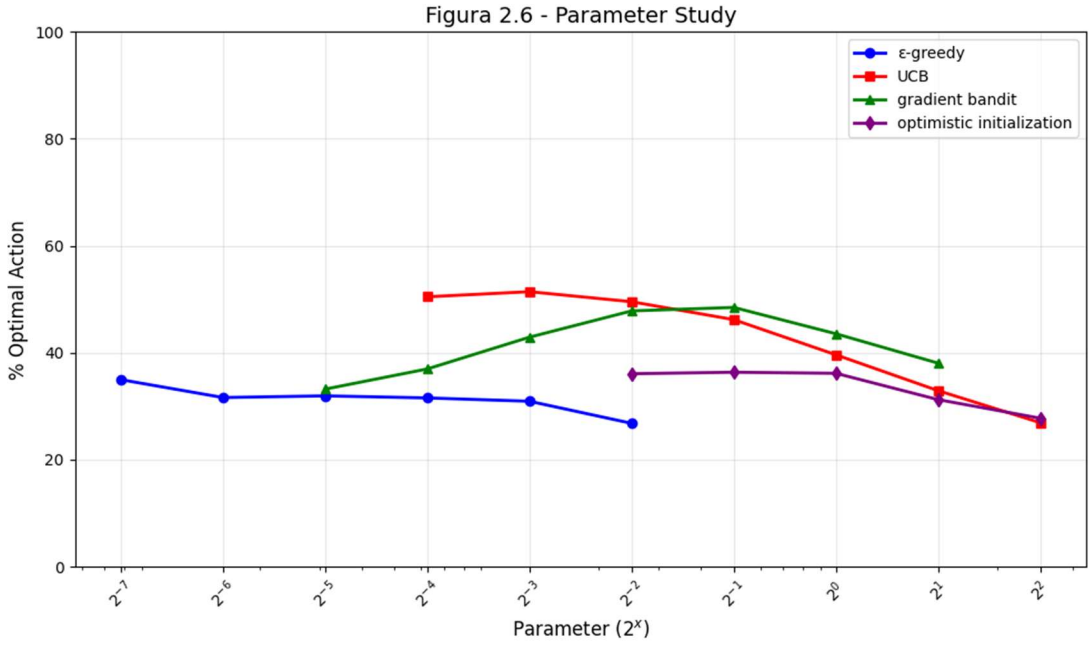


Figura 11- Variação de Parâmetros – Ambiente Não Estacionário

A Figura 12 apresenta a comparação entre o agente UCB ($c = 1/8$), o agente ε-greedy ($\varepsilon = 0{,}1$) e o agente *Thompson Sampling* para 1.000 passos e 2.000 repetições (*runs*) em um ambiente não estacionário. Nesse cenário, o agente UCB superou o *Thompson Sampling* em desempenho.

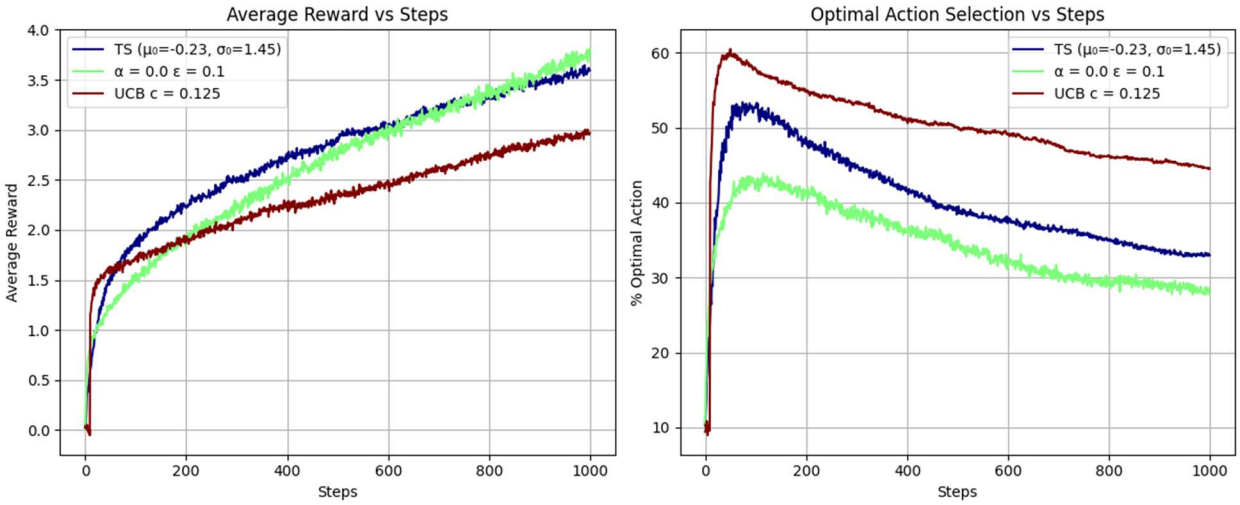


Figura 13- UCB com c = 1 e TS – Ambiente Não Estacionário

A Figura 13 apresenta o caso de teste de variação de $\alpha \in \{0{,}01; 0{,}1;\ 0{,}5\}$ com ε fixo e igual a 0,1 para o ambiente não estacionário. Nota-se que a estratégia de utilizar um valor maior de α apresenta melhor desempenho nos passos iniciais; contudo, o resultado deteriora-se à medida que o efeito da não estacionariedade do ambiente se acumula com o aumento do número de passos.

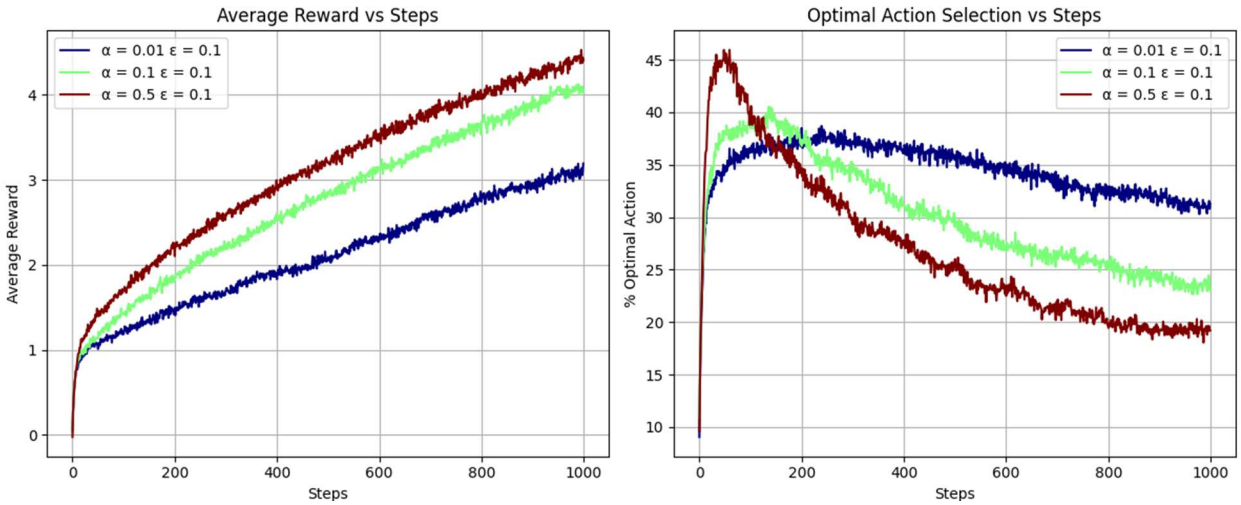


Figura 12- Efeito da variação de α -Ambiente Não Estacionário

7. **Conclusão:**

A solução desenvolvida em Python foi validada por meio da comparação com os resultados do livro de Sutton & Barto [1] e pode ser utilizada de maneira flexível para avaliar o desempenho dos agentes estudados, tanto em ambientes estacionários quanto não estacionários, em diversas outras configurações.

O algoritmo UCB mostrou-se uma solução de excelente custo-benefício para o cenário de ambiente estacionário.

Todos os agentes testados apresentaram baixo desempenho em ambientes não estacionários, indicando a necessidade de empregar técnicas específicas para esse tipo de cenário, tais como *Sliding-Window UCB*, *Discounted UCB* e agentes com memória adaptativa. A referência [2] apresenta uma descrição teórica mais aprofundada de diversas técnicas de solução para o problema dos K-Bandidos que não foram abordadas no presente artigo.

Fica registrado o agradecimento à Google pelo fornecimento do ambiente Google Colab com acesso a GPU para edição, testes e análise de código via Gemini integrado, bem como às IAs Gemini e DeepSeek pelas valiosas contribuições na revisão do texto, definição da arquitetura do *software*, elaboração e correção do código em Python e discussão dos resultados. Agradece-se também à IA Claude pela indicação de referências bibliográficas sobre aplicações reais do problema dos K-Armados Bandidos (*Multi-Armed Bandits*).

.

Referências:

[1] [Sutton & Barto] *Reinforcement Learning – Second Edition* – The MIT Press - Capítulo 2 - 2018

[2] [Lattimore & Szepesvári] *Bandit Algorithms* – Free Edition – Cambridge Press - 2020

[3] One-armed Bandit, Wiktionary, https://en.wiktionary.org/wiki/one-armed_bandit, acesso em 29/04/2026.

[4] [Bouneffouf & Rish] *A Survey on Practical Applications of Multi-Armed and Contextual Bandits* 02/04/2019

[5] *Artwork Personalization at Netflix, Netflix Blog*, https://netflixtechblog.com/artwork-personalization-c589f074ad76, 07/12/2017, acesso em 30/04/26

[6] [Amrallah, Mohamed, Tran & Sakagushi], MDPI, *UAV Trajectory Optimization in a Post-Disaster Area Using Dual-Energy-Aware Bandits*

[7] [Lapam] *Deep Reinforcement Learning Hands-On -Third Edition* – Capítulo 2 - 2024

[8] [Gamma, Helm, Johnson, Vlissides] *Design Patterns, Elements of Reusable Object-Oriented Software* – Páginas 315 e 325 – 1994

[9] GitHub - https://github.com/kwahcarioca/Reinforcement-Learning/tree/main/K-bandidos/notebooks